

Guía de Despliegue en Render

Proyecto Grocery Chef — Electiva de Backend

Introducción

Este documento describe el proceso completo para desplegar la infraestructura del proyecto Grocery Chef en la plataforma Render, compuesta por dos componentes independientes: una base de datos PostgreSQL y un servidor backend desarrollado en Node.js con Express, empaquetado como imagen Docker. El procedimiento sigue una estructura equivalente a la guía de despliegue en Azure provista por el docente, adaptada a las herramientas y flujo de trabajo de Render.

Pre-requisitos

Antes de iniciar, verificar que se cumplen los siguientes requisitos:

- Cuenta activa en Render (<https://render.com>) — el registro es gratuito.
- Docker instalado y funcionando en el equipo local.
- Acceso de colaborador al repositorio del proyecto en GitHub.
- Archivo **render.yaml** ubicado en la raíz del repositorio (se crea en el Paso 4).
- Archivo **database.ts** actualizado con soporte SSL (se detalla en el Paso 3).

1 Instalación del Render CLI

El Render CLI es la herramienta de línea de comandos oficial que permite gestionar servicios, consultar logs, ejecutar redeloys y acceder a bases de datos directamente desde la terminal. Para instalarlo en Linux Mint, ejecutar el siguiente comando:

```
curl -fsSL
https://raw.githubusercontent.com/render-oss/cli/refs/heads/main/bin/install.sh |
sh
```

El instalador descarga automáticamente la versión más reciente y ubica el binario en **~/.local/bin/render**. Para verificar que la instalación fue exitosa:

```
render --version

# Salida esperada: render v2.12.0 (o superior)
```

Si al ejecutar render el sistema responde «orden no encontrada», es posible que **~/.local/bin** no esté incluido en el PATH. Ejecutar: `echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc && source ~/.bashrc`

2 Autenticación y selección de workspace

Para vincular el CLI con la cuenta de Render, ejecutar el siguiente comando. El proceso abre automáticamente el navegador para completar la autenticación:

```
render login
```

En la página que se abre en el navegador, hacer click en **Generate token**. Una vez generado, regresar a la terminal donde el CLI confirmará el inicio de sesión y solicitará seleccionar el workspace activo. Para listar los workspaces disponibles y seleccionar el correcto:

```
render workspaces
```

3 Actualización de database.ts para soporte SSL

Render exige que las conexiones a PostgreSQL se realicen mediante SSL. Sin esta configuración el backend no puede establecer conexión con la base de datos y el servicio fallará al arrancar. Modificar el archivo **backend/src/config/database.ts** para que quede de la siguiente manera:

```
import { Sequelize } from 'sequelize';

import dotenv from 'dotenv';

dotenv.config();

const sequelize = new Sequelize(

  process.env.DB_NAME as string,

  process.env.DB_USER as string,

  process.env.DB_PASSWORD as string,

  {

    host: process.env.DB_HOST,

    port: Number(process.env.DB_PORT) || 5432,

    dialect: 'postgres',

    logging: false,

    dialectOptions: {
```

```
// Activar SSL únicamente cuando la variable DB_SSL sea 'true'

ssl: process.env.DB_SSL === 'true'

? { rejectUnauthorized: false }

: false,

},

}

);

export default sequelize;
```

Importante: omitir esta actualización impedirá que el backend establezca conexión con la base de datos en producción.

4 Creación del archivo render.yaml

El archivo render.yaml es el equivalente al script de infraestructura de Azure utilizado en la guía del docente. Define de forma declarativa los dos recursos que Render debe crear: la base de datos PostgreSQL y el servicio web del backend. Este archivo debe ubicarse en la **raíz del repositorio**. A continuación el contenido completo del archivo:

```
# render.yaml - ubicar en la RAÍZ del repositorio

services:

- type: web

name: recetario-backend

runtime: docker

rootDir: backend # subcarpeta del repo donde está el Dockerfile

dockerfilePath: ./Dockerfile

plan: free

region: virginia

healthCheckPath: /api/health
```

```
envVars:

- key: PORT

value: 3000

- key: NODE_ENV

value: production

- key: DB_HOST

fromDatabase:

name: recetario-db

property: host

- key: DB_PORT

fromDatabase:

name: recetario-db

property: port

- key: DB_USER

fromDatabase:

name: recetario-db

property: user

- key: DB_PASSWORD

fromDatabase:

name: recetario-db

property: password

- key: DB_NAME

fromDatabase:

name: recetario-db
```

```
property: database

- key: DB_SSL

value: 'true'

- key: JWT_SECRET

generateValue: true # Render genera un valor aleatorio seguro


databases:

- name: recetario-db

plan: free

region: virginia

databaseName: recetario_db

user: recetario_user

postgresMajorVersion: 15
```

Las credenciales de la base de datos (host, puerto, usuario, contraseña) son generadas automáticamente por Render e inyectadas en el backend mediante la directiva `fromDatabase`. No es necesario copiarlas ni configurarlas manualmente. El valor de `JWT_SECRET` también es generado por Render y se puede consultar en el dashboard bajo Environment del servicio `recetario-backend`.

5 Publicación de los cambios en el repositorio

Una vez creados y verificados los archivos `render.yaml` y `database.ts`, publicar los cambios en el repositorio remoto de GitHub:

```
# Ejecutar desde la raíz del repositorio

git add render.yaml

git add backend/src/config/database.ts

git commit -m 'add render deploy config + ssl support'

git push
```

6 Creación de los servicios en Render — Blueprint

Render utiliza el concepto de Blueprint para crear múltiples servicios a partir de un archivo `render.yaml`. Dado que el repositorio pertenece a otro colaborador, se utiliza la opción de repositorio público en lugar de la integración OAuth de GitHub. Seguir los pasos a continuación:

- 6.1 Ingresar al dashboard en <https://dashboard.render.com>
- 6.2 Hacer click en el botón **New** (esquina superior derecha)
- 6.3 Seleccionar la opción **Blueprint**
- 6.4 Elegir **Public Git Repository** e ingresar la URL del repositorio
- 6.5 En el campo Blueprint Name ingresar: **grocery-chef**
- 6.6 Render detecta el `render.yaml` automáticamente — hacer click en **Apply**
- 6.7 Render inicia la creación de la base de datos (~2 minutos) y del backend (~5 minutos)
- 6.8 Cuando ambos servicios muestren el estado **Live**, el despliegue está completo

7 Verificación del despliegue

Una vez que el dashboard muestre ambos servicios en estado Live, confirmar el correcto funcionamiento desde la terminal:

```
# Listar todos los servicios del workspace con sus IDs y URLs

render services -o text


# Verificar que el backend responde correctamente

curl https://recetario-backend.onrender.com/api/health


# Respuesta esperada:

# { "message": "Grocery Chef API is running!", "status": "ok" }
```

En el plan gratuito de Render, el backend entra en estado de inactividad tras 15 minutos sin recibir solicitudes.

8 Redeploy manual ante cambios en el código

Al ser colaborador del repositorio, la función de Auto-Deploy automático no está disponible. Por lo tanto, cada vez que se publiquen cambios mediante git push, es necesario ejecutar un redeploy manual desde el CLI. El procedimiento es el siguiente:

```
# 1. Obtener el SERVICE_ID del backend

render services -o text

# Identificar la línea correspondiente a recetario-backend

# El ID tiene el formato: srv-xxxxxxxxxxxxx

# 2. Ejecutar el redeploy (--wait mantiene el proceso activo hasta que finalice)

render deploys create --wait

# Ejemplo con un ID real:

render deploys create srv-abc123xyz --wait

# Para consultar el historial de deploys de un servicio:

render deploys list
```

El flag `--wait` es recomendable porque mantiene el proceso activo en la terminal hasta que el deploy finaliza y reporta si fue exitoso o si hubo errores.

9 Acceso directo a la base de datos (opcional)

En caso de necesitar inspeccionar la base de datos directamente — por ejemplo, para verificar que las tablas fueron creadas correctamente por `sync.ts` — el CLI permite abrir una sesión `psql` sin necesidad de conocer las credenciales:

```
# Obtener el DATABASE_ID de la base de datos

render services -o text

# El ID tiene el formato: dpg-xxxxxxxxxxxxx
```

```
# Abrir una sesión psql interactiva
```

```
render psql
```

```
# Ejecutar una consulta puntual sin abrir sesión interactiva
```

```
render psql -c '\dt' # listar tablas
```
